

## Q30. Application: Cybersecurity Threat Analysis System

### Aim

To solve the recurrence relation

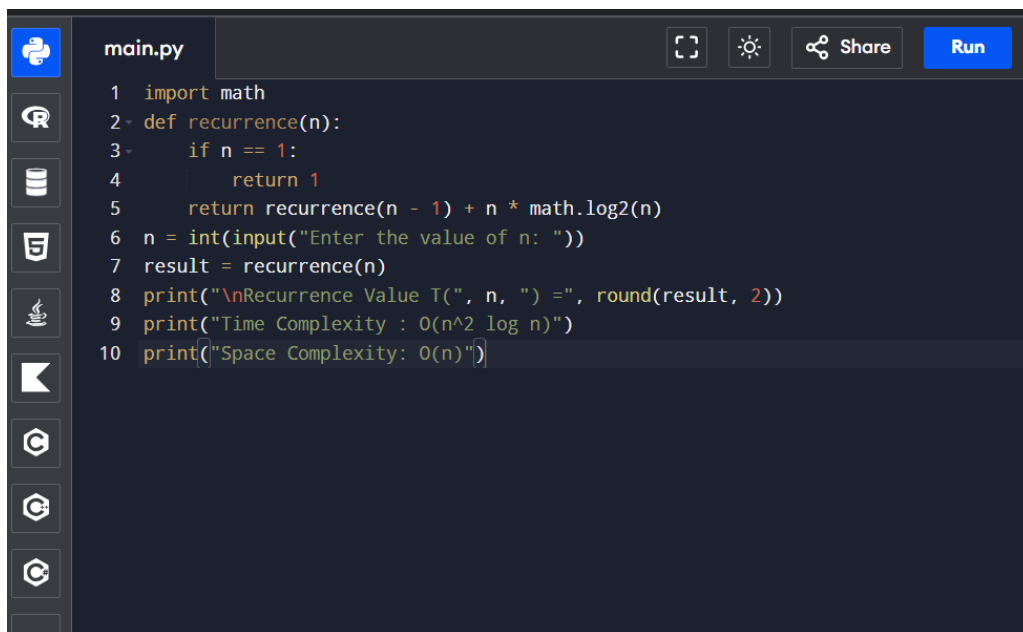
$$T(n) = T(n - 1) + n \log n$$

using the substitution method, derive its time complexity, and analyse its scalability in cybersecurity threat analysis.

### Algorithm

1. Start with the given recurrence relation.
2. Expand the recurrence repeatedly using substitution.
3. Continue until the base case  $T(1)$ .
4. Convert the expansion into summation form.
5. Evaluate the summation.
6. Derive the asymptotic time complexity.
7. Analyse scalability using asymptotic notation.

### Source code



```
main.py
1 import math
2 def recurrence(n):
3     if n == 1:
4         return 1
5     return recurrence(n - 1) + n * math.log2(n)
6 n = int(input("Enter the value of n: "))
7 result = recurrence(n)
8 print("\nRecurrence Value T(", n, ") =", round(result, 2))
9 print("Time Complexity : O(n^2 log n)")
10 print("Space Complexity: O(n)")
```

### Step-by-Step Analysis

Given,

$$T(n) = T(n - 1) + n \log n$$

Expand once,

$$T(n) = T(n - 2) + (n - 1) \log(n - 1) + n \log n$$

Expand again,

$$T(n) = T(n - 3) + (n - 2) \log(n - 2) + (n - 1) \log(n - 1) + n \log n$$

Continuing until the base case,

$$T(n) = T(1) + \sum_{k=2}^n k \log k$$

Since,

$$\sum_{k=1}^n k \log k = \Theta(n^2 \log n)$$

Therefore,

$$\boxed{T(n) = \Theta(n^2 \log n)}$$

### Time Complexity

$$\boxed{\Theta(n^2 \log n)}$$

### Space Complexity

$$\boxed{O(n)}$$

## Scalability Analysis

- As the number of cybersecurity threats increases, each new threat requires additional analysis.
- The running time grows quadratically with an extra logarithmic factor.
- For very large threat datasets, execution time increases significantly.
- Suitable for medium-sized datasets but becomes expensive for extremely large-scale cybersecurity systems.

## Asymptotic Interpretation

- Best Case:  $O(n^2 \log n)$
- Average Case:  $\Theta(n^2 \log n)$
- Worst Case:  $\Omega(n^2 \log n)$

Hence,

$$T(n) = \Theta(n^2 \log n)$$

## Sample input and output

### Output

```
Enter the value of n: 5
```

```
Recurrence Value T( 5 ) = 27.36
```

```
Time Complexity :  $O(n^2 \log n)$ 
```

```
Space Complexity:  $O(n)$ 
```

```
=== Code Execution Successful ===
```

## Q31. Application: E-Commerce Recommendation System

### Aim

To solve the recurrence relation

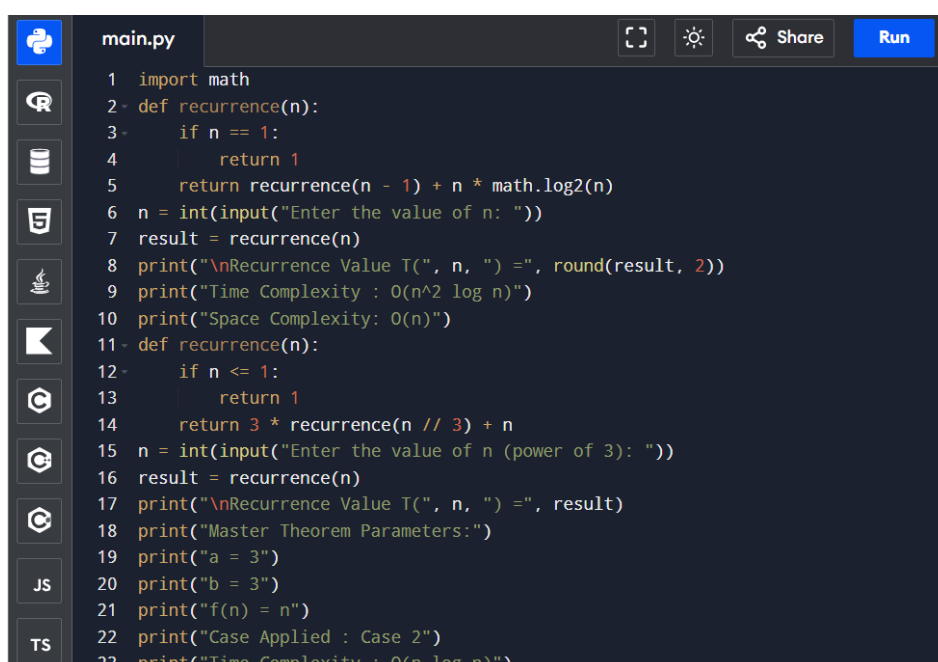
$$T(n) = 3T(n/3) + n$$

using the Master Theorem, derive its time complexity, and analyse its scalability in an e-commerce recommendation system.

### Algorithm

1. Identify the recurrence relation.
2. Compare it with the Master Theorem form.
3. Determine the values of  $a$ ,  $b$ , and  $f(n)$ .
4. Compute  $n^{\log_b a}$ .
5. Select the appropriate Master Theorem case.
6. Derive the asymptotic complexity.
7. Analyse scalability using asymptotic notation.

### Source code



```
main.py
1 import math
2 def recurrence(n):
3     if n == 1:
4         return 1
5     return recurrence(n - 1) + n * math.log2(n)
6 n = int(input("Enter the value of n: "))
7 result = recurrence(n)
8 print("\nRecurrence Value T(", n, ") =", round(result, 2))
9 print("Time Complexity : O(n^2 log n)")
10 print("Space Complexity: O(n)")
11 def recurrence(n):
12     if n <= 1:
13         return 1
14     return 3 * recurrence(n // 3) + n
15 n = int(input("Enter the value of n (power of 3): "))
16 result = recurrence(n)
17 print("\nRecurrence Value T(", n, ") =", result)
18 print("Master Theorem Parameters:")
19 print("a = 3")
20 print("b = 3")
21 print("f(n) = n")
22 print("Case Applied : Case 2")
23 print("Time Complexity : O(n log n)")
```

### Step-by-Step Analysis

Given,

$$T(n) = 3T\left(\frac{n}{3}\right) + n$$

Compare with the standard Master Theorem:

$$T(n) = aT(n/b) + f(n)$$

Here,

- $a = 3$
- $b = 3$
- $f(n) = n$

Now,

$$\log_b a = \log_3 3 = 1$$

Therefore,

$$n^{\log_b a} = n^1 = n$$

Since,

$$f(n) = \Theta(n) = \Theta(n^{\log_b a})$$

This satisfies Master Theorem Case 2.

Hence,

$$T(n) = \Theta(n \log n)$$

### Master Theorem Justification

- $a = 3$
- $b = 3$
- $f(n) = n$
- $n^{\log_3 3} = n$
- Since  $f(n) = \Theta(n^{\log_b a})$ ,

Master Theorem Case 2 applies.

Therefore,

$$T(n) = \Theta(n \log n)$$

### Time Complexity

$$\Theta(n \log n)$$

### Space Complexity

Recursive height:

$$\log_3 n$$

Therefore,

$$O(\log n)$$

### Scalability Analysis

- The recommendation engine divides users into three equal groups recursively.
- Each level processes all users once.
- The recursion depth is only  $\log_3 n$ , making the algorithm highly scalable.
- As the number of users increases, execution time grows almost linearly with a logarithmic factor.
- This makes it efficient for handling millions of users in large e-commerce platforms.

### Asymptotic Interpretation

- Best Case:  $O(n \log n)$
- Average Case:  $\Theta(n \log n)$
- Worst Case:  $\Omega(n \log n)$

Hence,

$$T(n) = \Theta(n \log n)$$

### Sample input and output

```
Output
Enter the value of n: 27

Recurrence Value T( 27 ) = 1536.17
Time Complexity :  $O(n^2 \log n)$ 
Space Complexity:  $O(n)$ 
Enter the value of n (power of 3): |
```